

A COOKBOOK FOR BACKEND DEVELOPERS

Redis Backend Cookbook

Master Redis for real backends — caching, locks, rate limiting, pub/sub, and streams — with Node.js and Python examples side by side.

Redis 7+

ioredis

redis-py

Caching

Locks

Streams

Rate limiting

Cluster

Written for engineers who know Node.js/Express (and are learning Python) and want Redis as production infrastructure.

Every pattern answers: what problem it solves, how to do it in Node and Python, the pitfalls, and when NOT to use it.

Edition 2026 · concepts · commands · production patterns

Table of Contents

How to Use This Book	4
The one idea that explains Redis	4
Single-threaded, and why that's a feature.	4
What you'll be able to build	5
Part 1 · The Redis Mental Model	6
1.1 Where Redis fits	6
1.2 Everything is a key	6
1.3 The atomicity guarantee	7
1.4 Latency, throughput, and the slow-command trap	7
Part 2 · Setup & Clients	9
2.1 Running Redis locally	9
2.2 Connecting from Node (ioredis)	9
2.3 Connecting from Python (redis-py)	10
2.4 Connection pooling and the blocking gotcha	10
Part 3 · Core Data Types	12
3.1 Strings — values, counters, flags	12
3.2 Hashes — objects / records	12
3.3 Lists — queues and stacks.	13
3.4 Sets — uniqueness and membership	13
3.5 Sorted sets — leaderboards, priority, time-ordering.	14
3.6 Bitmaps and HyperLogLog — counting at scale	14
Part 4 · Keys, Expiration & Eviction	16
4.1 TTL — time to live	16
4.2 How expiration actually works.	16
4.3 Eviction — what happens when memory is full	16
4.4 Iterating keys safely	17
Part 5 · Caching Patterns	19
5.1 Cache-aside (lazy loading) — the default	19
5.2 Invalidation — the hard part.	20
5.3 Cache stampede (thundering herd)	20
5.4 What to cache — and what not to	21
Part 6 · Distributed Locks	22
6.1 The correct simple lock	22
6.2 Why every piece matters	23
6.3 TTL, renewal, and honesty about guarantees	23
Part 7 · Rate Limiting	25
7.1 Fixed window — simplest	25
7.2 Sliding window — smooth and fair.	25
7.3 Token bucket — allow bursts, cap the rate	26
Part 8 · Pub/Sub and Streams	27

8.1 Pub/Sub — broadcast, no durability	27
8.2 Streams — a durable, replayable log	28
Part 9 • Transactions & Lua Scripting	30
9.1 MULTI/EXEC — queue then run atomically	30
9.2 WATCH — optimistic concurrency	30
9.3 Lua scripts — atomic read-compute-write	31
Part 10 • Persistence & Durability	33
10.1 RDB vs AOF.	33
10.2 The honest durability story.	33
10.3 Choosing per use case	34
10.4 Memory is the real constraint	34
Part 11 • Production Redis	35
11.1 High availability: replication, Sentinel, Cluster	35
11.2 Security	35
11.3 Observability	36
11.4 The production mistakes that bite	36
Part 12 • Patterns & Commands Cheat Sheet	37
12.1 Data type → what it's for	37
12.2 Commands by task.	37
12.3 Node ↔ Python ↔ CLI.	38
12.4 The pattern recipes (from this book)	38
12.5 The production checklist	38

How to Use This Book

You already build backends: you cache things, you run background work, you rate-limit endpoints. This book makes **Redis** the tool you reach for to do all of that well. It is written for an engineer fluent in Node.js (and picking up Python), so every pattern is shown twice — once with **ioredis** (Node) and once with **redis-py** (Python) — plus the raw `redis-cli` commands underneath.

This is not a command reference; it is a *patterns* book. Each chapter answers the same four questions:

1. **What problem does this solve?**
2. **How do I do it in Node and in Python?**
3. **What are the pitfalls?**
4. **When should I *not* use Redis for this?**

The one idea that explains Redis

Redis is an **in-memory data-structure server**. Two words carry all the weight. *In-memory*: your data lives in RAM, so operations are measured in microseconds, and memory is the resource you budget and can run out of. *Data-structure server*: it is not a key→string blob store — it speaks native strings, hashes, lists, sets, sorted sets, streams, and more, with atomic operations on each. Almost everything Redis is good at (caching, locks, counters, rate limiters, queues, leaderboards, pub/sub) is just picking the right data structure and leaning on the fact that **each command is atomic**.

NODE.JS MENTAL MODEL

Think of Redis as a shared, networked version of the JavaScript primitives you already use — a `Map` (hash), an `Array` (list), a `Set`, a sorted set (like a priority queue) — living in one fast process every instance of your app can reach. In Node you'd keep a `Map` in module scope, but it dies with the process and isn't shared across instances. Redis is that `Map`, made durable-enough, atomic, and shared across your whole fleet.

Single-threaded, and why that's a feature

Redis executes commands on **one thread**, one at a time. That sounds like a limitation but is the source of its greatest strength: **every command is atomic** with no locks to manage, because nothing else runs while it does. A `INCR` can never race another `INCR`. This is why Redis is the natural home for counters, locks, and rate limiters. The flip side: one slow command (a `KEYS *` on millions of keys, a big `SORT`) blocks *everyone*, so "don't run slow commands" is a rule you'll meet throughout this book.

What you'll be able to build

By the end you can confidently use Redis for: a cache that doesn't stampede; distributed locks that don't deadlock; rate limiters that actually hold under load; real-time messaging with pub/sub and streams; atomic multi-step operations with transactions and Lua; and a production deployment with the right persistence, eviction, and high-availability choices. The final chapter is a patterns-and-commands cheat sheet to keep at your desk.

A companion **Backend Infra Sprint** plan and two sibling cookbooks (Queues, Mailpit) round out the set — Redis is the foundation the other two build on.

Part 1 · The Redis Mental Model

Before commands, get the model right — it prevents most misuse. Redis is a single-threaded, in-memory, key-addressed data-structure server. This part turns that sentence into intuition.

1.1 Where Redis fits

Redis is not a replacement for your primary database (Postgres/MySQL). It sits *beside* it as a fast, ephemeral-leaning layer for data that is hot, derived, or transient.

Job	Primary DB (Postgres)	Redis
Source of truth	✓ durable, relational	✗ (treat as losable)
Hot reads / caching	slow-ish	✓ microseconds
Counters / rate limits	row locks, contention	✓ atomic <code>INCR</code>
Locks / coordination	advisory locks	✓ <code>SET NX</code> / Redlock
Queues / streams	possible, awkward	✓ lists / streams
Real-time pub/sub	✗	✓
Leaderboards / ranking	<code>ORDER BY</code> , heavy	✓ sorted sets
Complex queries / joins	✓	✗

WHEN NOT TO USE REDIS

Redis is the wrong home for your system of record. If losing the data would be a real incident, it belongs in a durable database; Redis persistence (Part 10) is a safety net, not a guarantee you should design around. Also avoid Redis for large blobs (megabyte values), rich queries, or anything you can't fit in RAM — memory is finite and expensive.

1.2 Everything is a key

Redis is one big keyspace. A key is a string; its *value* is one of several data types. Good key naming is your schema — use a consistent, colon-namespaced convention.

SHELL

```
SET user:42:name "Ahmed"           # string
HSET user:42 name "Ahmed" age 31   # hash (an object)
EXPIRE user:42 3600                 # keys can auto-expire (TTL)
```

Conventions that scale: `entity:id:field` (`user:42:email`), and a per-domain prefix (`cache:`, `lock:`, `rl:`, `queue:`). Consistent prefixes make it possible to reason about, monitor, and safely bulk-invalidate keys.

1.3 The atomicity guarantee

Because one thread runs commands sequentially, each single command is atomic. No two clients can interleave inside a command.

JAVASCRIPT / NODE.JS

```
// ioredis (Node) – safe under any concurrency, no locks needed
await redis.incr("page:views");           // atomic increment
await redis.hincrby("user:42", "logins", 1);
```

PYTHON

```
# redis-py (Python)
r.incr("page:views")
r.hincrby("user:42", "logins", 1)
```

This is the whole reason counters, locks, and rate limiters are trivial in Redis and fiddly in a relational database. When you need *several* commands to be atomic together, you use transactions or Lua (Part 9) — but reach for a single atomic command first.

COMMON MISTAKE

The classic race is read-modify-write in your app: `GET` a value, change it in Node/Python, `SET` it back. Between your `GET` and `SET`, another process can change it — you clobber their write. Use an atomic command (`INCR`, `HINCRBY`, `SETNX`) or a Lua script instead of round-tripping the value through your app.

1.4 Latency, throughput, and the slow-command trap

A Redis op on localhost is tens of microseconds; over a network, sub-millisecond plus round-trip. A single instance handles ~100k+ ops/sec easily. The limiter is usually **round trips** (chatty code) and **slow commands** (which block the one thread).

JAVASCRIPT / NODE.JS

```
// SLOW: N round trips
for (const id of ids) await redis.get(`user:${id}`); // N network hops

// FAST: one round trip with a pipeline
const pipe = redis.pipeline();
ids.forEach((id) => pipe.get(`user:${id}`));
const results = await pipe.exec(); // one hop
```

PRODUCTION TIP

Two habits pay off immediately: (1) **pipeline** or use multi-key commands (`MGET` , `HMGET`) to collapse many round trips into one; (2) **never run $O(N)$ commands on big keyspaces in production** — `KEYS *` , `FLUSHALL` on a live DB, `SMEMBERS` on a huge set. Use `SCAN` (cursor-based, non-blocking) to iterate keys instead of `KEYS` .

EXERCISE 1.1

Model a "user session + counters" in Redis by hand in `redis-cli` : a hash `user:1` with fields, a TTL of one hour, and an atomic `HINCRBY` for a login counter. Then write the same in both `ioredis` and `redis-py`, and rewrite a loop of single `GET` s as one pipeline; confirm (via `redis-cli --latency` or timing) the round-trip difference.

Part 2 · Setup & Clients

You will run Redis locally (Docker), talk to it with `redis-cli`, and connect from Node (ioredis) and Python (redis-py). This part gets all three working and covers the connection concerns that bite in production.

2.1 Running Redis locally

SHELL

```
# one-off container
docker run -d --name redis -p 6379:6379 redis:7-alpine

# or in docker-compose.yml
# services:
#   redis:
#     image: redis:7-alpine
#     ports: ["6379:6379"]
#     command: ["redis-server", "--appendonly", "yes"] # enable AOF (Part 10)
#     volumes: ["redis-data:/data"]

redis-cli ping # -> PONG
redis-cli # interactive shell
127.0.0.1:6379> set hello world
127.0.0.1:6379> get hello
```

`redis-cli` is your microscope: `MONITOR` (watch every command live — dev only), `--latency`, `INFO`, `SLOWLOG GET`, `DBSIZE`, `MEMORY USAGE key`.

2.2 Connecting from Node (ioredis)

JAVASCRIPT / NODE.JS

```
import Redis from "ioredis";

const redis = new Redis({
  host: process.env.REDIS_HOST ?? "127.0.0.1",
  port: 6379,
  maxRetriesPerRequest: 3,
  enableReadyCheck: true,
});
// or: const redis = new Redis(process.env.REDIS_URL); // redis://...

redis.on("error", (e) => logger.error({ e }, "redis error"));
```

```
await redis.set("k", "v", "EX", 60);    // SET k v EX 60
const v = await redis.get("k");
```

2.3 Connecting from Python (redis-py)

PYTHON

```
import redis

# decode_responses=True returns str instead of bytes – usually what you want
pool = redis.ConnectionPool(host="127.0.0.1", port=6379, decode_responses=True)
r = redis.Redis(connection_pool=pool)

r.set("k", "v", ex=60)           # SET k v EX 60
v = r.get("k")

# async variant for FastAPI etc.
import redis.asyncio as aredis
ar = aredis.from_url("redis://localhost", decode_responses=True)
# await ar.get("k")
```

Concern	ioredis (Node)	redis-py (Python)
Create client	<code>new Redis(url)</code>	<code>redis.from_url(url)</code>
Connection pool	built in	<code>ConnectionPool</code> (sync) / async pool
Return type	strings	bytes, or <code>str</code> with <code>decode_responses=True</code>
Pipeline	<code>redis.pipeline()</code>	<code>r.pipeline()</code>
Pub/sub	<code>redis.duplicate()</code>	<code>r.psubsub()</code>
Async	native promises	<code>redis.asyncio</code>

2.4 Connection pooling and the blocking gotcha

Both clients pool connections. The rule to remember: **a connection running a blocking command (BLPOP, SUBSCRIBE, XREAD BLOCK, WAIT) is busy and can't serve other commands.** Use a *dedicated* connection for subscriptions and blocking reads, not your shared pool.

JAVASCRIPT / NODE.JS

```
const sub = redis.duplicate();    // separate connection for pub/sub
sub.subscribe("events");
sub.on("message", (chan, msg) => handle(msg));
// keep `redis` free for normal commands
```

COMMON MISTAKE

Calling `SUBSCRIBE` or a blocking pop on your main client and then wondering why normal commands hang — that connection is now dedicated to the subscription. Always `duplicate()` (ioredis) or open a separate client for blocking/pub-sub work. Also: create the client **once** at startup and reuse it; opening a new connection per request exhausts sockets under load.

PRODUCTION TIP

Set connection **timeouts** and a bounded **retry/backoff** so a Redis blip fails fast instead of hanging requests, and always attach an `error` handler (an unhandled Redis error can crash a Node process). In serverless/edge environments where you can't hold a socket, use a Redis provider's HTTP API or a proxy — long-lived TCP pools don't fit that model.

EXERCISE 2.1

Stand up Redis in docker-compose with AOF on and a named volume. Connect from both ioredis and redis-py, set a key with a 60s TTL, and read it back. Then open a `SUBSCRIBE` on a duplicated connection and publish to it from `redis-cli`, confirming your main client still answers `GET / SET` while the subscription is active.

Part 3 · Core Data Types

Redis is only as good as your choice of data structure. Pick the right one and the operation you need is a single atomic command; pick wrong and you fight the tool. This part tours the workhorses with the problem each one solves.

3.1 Strings — values, counters, flags

The simplest type: a key holds a string (or a number, or bytes up to 512 MB). Strings back caching, counters, feature flags, and simple locks.

SHELL

```
SET session:abc "..." EX 3600      # value with a 1-hour TTL
INCR page:views                      # atomic counter (creates at 0)
INCRBY credits:42 10
SETNX lock:job "1"                  # set only if absent (a naive lock)
MGET a b c                          # multi-get in one round trip
```

JAVASCRIPT / NODE.JS

```
await redis.set("page:views", 0);
await redis.incr("page:views");      // 1
await redis.set("flag:beta", "on", "EX", 300);
```

PYTHON

```
r.incr("page:views")
r.set("flag:beta", "on", ex=300)
```

3.2 Hashes — objects / records

A hash is a map of field→value under one key — a JavaScript object or a small record. Use it to store an entity's fields without JSON-encoding the whole thing (you can read/update single fields atomically).

SHELL

```
HSET user:42 name "Ahmed" age 31 plan "pro"
HGET user:42 plan
HINCRBY user:42 logins 1           # atomic field increment
HGETALL user:42                   # the whole object (small hashes only)
```

JAVASCRIPT / NODE.JS

```
await redis.hset("user:42", { name: "Ahmed", age: 31 });
const plan = await redis.hget("user:42", "plan");
const user = await redis.hgetall("user:42");    // { name, age, plan }
```

NODE.JS MENTAL MODEL

A hash is your object stored field-addressably: instead of `SET user:42 JSON.stringify(user)` (which forces you to read+parse+write the whole object to change one field), `HSET / HGET / HINCRBY` let you touch one field atomically. Prefer a hash over a JSON string whenever you update parts of an object independently.

3.3 Lists — queues and stacks

A list is an ordered sequence you push/pop from either end — a deque. It's the basis of simple job queues (Part covered in the Queues book) and recent-activity feeds.

SHELL

```
LPUSH queue:emails "job1"           # push left
RPOPLPUSH queue:emails queue:proc    # atomically move (reliable queue)
BRPOP queue:emails 5                 # blocking pop, wait up to 5s (worker)
LRANGE feed:42 0 9                   # newest 10 items
LTRIM feed:42 0 99                    # cap the list to 100
```

JAVASCRIPT / NODE.JS

```
await redis.lpush("feed:42", item);
await redis.ltrim("feed:42", 0, 99);           // keep last 100
const recent = await redis.lrange("feed:42", 0, 9);
```

3.4 Sets — uniqueness and membership

An unordered collection of unique members. Perfect for "have we seen this?", tags, unique visitors, and set math (intersections, unions).

SHELL

```
SADD seen:today "user:42"
SISMEMBER seen:today "user:42"    # 0(1) membership test
SCARD seen:today                   # count of unique members
SINTER tags:redis tags:python      # items tagged both
```

PYTHON

```
r.sadd("online", user_id)
if r.sismember("online", user_id): ...
count = r.scard("online")
```

3.5 Sorted sets — leaderboards, priority, time-ordering

A set where every member has a **score**; members are kept ordered by score. This one type powers leaderboards, priority queues, rate limiters (Part 7), delayed jobs, and "top N" — anything ranked or time-ordered.

SHELL

```
ZADD leaderboard 1500 "ahmed" 1800 "sara"
ZINCRBY leaderboard 50 "ahmed"      # bump a score atomically
ZREVRANGE leaderboard 0 9 WITHSCORES # top 10
ZRANK leaderboard "ahmed"           # a player's rank
ZRANGEBYSCORE due 0 1699999999      # items due before a timestamp (delayed jobs)
```

JAVASCRIPT / NODE.JS

```
await redis.zadd("leaderboard", 1500, "ahmed");
const top = await redis.zrevrange("leaderboard", 0, 9, "WITHSCORES");
```

PRODUCTION TIP

Sorted sets are the Swiss-army structure. Using the **score as a timestamp** turns a sorted set into a time index: schedule delayed work by `ZADD due <run_at> job`, then a worker polls `ZRANGEBYSCORE due 0 <now>` and `ZREM`s what it takes. The same trick gives you sliding-window rate limiting (Part 7) and time-bucketed feeds.

3.6 Bitmaps and HyperLogLog — counting at scale

Two specialized types for space-efficient analytics:

- **Bitmaps** (operations on string bits): track per-user boolean state (e.g. "did user N do X today?") in a handful of bytes. `SETBIT active:2026-01-01 42 1`; `BITCOUNT` counts the set bits.
- **HyperLogLog**: estimate the number of *unique* items using ~12 KB regardless of cardinality, with ~0.8% error. `PFADD visitors:today user42`; `PFCOUNT visitors:today`. Use it for unique-visitor counts where exactness doesn't matter.

You need	Use	Key command
A value / counter / flag	string	<code>SET</code> / <code>INCR</code>
An object with fields	hash	<code>HSET</code> / <code>HINCRBY</code>
A queue / recent feed	list	<code>LPUSH</code> / <code>BRPOP</code> / <code>LTRIM</code>
Uniqueness / membership	set	<code>SADD</code> / <code>SISMEMBER</code>
Ranking / time-ordering / priority	sorted set	<code>ZADD</code> / <code>ZRANGEBYSCORE</code>
Per-user boolean at scale	bitmap	<code>SETBIT</code> / <code>BITCOUNT</code>
Approx unique count, tiny memory	HyperLogLog	<code>PFADD</code> / <code>PFCOUNT</code>

You need	Use	Key command
Streaming / event log	stream	XADD / XREADGROUP (Part 8)

COMMON MISTAKE

Storing everything as JSON strings and losing Redis's superpower. If you `SET user:42 '{"...}'` you can't atomically bump one field, rank users, or test membership — you've turned a data-structure server into a dumb blob store. Model with the type that matches the *access pattern*: rank → sorted set, membership → set, partial updates → hash.

EXERCISE 3.1

Build a mini leaderboard: `ZADD` ten players with scores, `ZINCRBY` a few, and fetch the top 5 with scores and one player's rank — in both `ioredis` and `redis-py`. Then track "unique visitors today" two ways (a set and a HyperLogLog) and compare `SCARD / PFCOUNT` and the memory each uses (`MEMORY USAGE`).

Part 4 · Keys, Expiration & Eviction

Redis lives in RAM, so *when data leaves* matters as much as how it goes in. Expiration (TTL) and eviction are how you keep memory bounded and caches fresh. Getting these two right is most of what separates a healthy Redis from one that falls over at 3 a.m.

4.1 TTL — time to live

Any key can be given an expiry, after which Redis deletes it automatically.

SHELL

```
SET session:abc "... " EX 3600      # expire in 3600s
EXPIRE user:42 300                  # set/replace TTL on an existing key
TTL session:abc                     # seconds left (-1 no TTL, -2 gone)
PERSIST session:abc                 # remove the TTL (make permanent)
SET k v KEeptTL                     # update value, keep existing TTL
```

JAVASCRIPT / NODE.JS

```
await redis.set("session:abc", data, "EX", 3600);
const ttl = await redis.ttl("session:abc");
```

COMMON MISTAKE

Two TTL traps. (1) Plain `SET key value` on an existing key **clears its TTL** — the key becomes permanent. Use `KEEPTTL` or re-`EXPIRE` if you meant to keep it. (2) Setting no TTL on cache keys, so your "cache" grows forever and eventually eats all memory. Cache entries should almost always have an expiry.

4.2 How expiration actually works

Redis expires keys two ways: **lazily** (when you access an expired key, it's removed then) and via a **background sampler** (it periodically checks a random sample of keys with TTLs). The practical consequence: an expired key may still occupy memory until it's touched or sampled. Don't assume memory frees the exact instant a TTL passes.

4.3 Eviction — what happens when memory is full

When Redis hits `maxmemory`, it applies an **eviction policy** to make room (or starts refusing writes). Choosing this policy is a required production decision.

SHELL


```
# redis.conf / redis-server flags
maxmemory 512mb
maxmemory-policy allkeys-lru      # for a cache: evict least-recently-used
```

Policy	Behavior	Use when
noeviction	reject writes when full (default)	Redis is a data store, not a cache
allkeys-lru	evict least-recently-used, any key	general cache
allkeys-lfu	evict least-frequently-used	cache with hot/cold skew
volatile-lru	LRU, but only keys with a TTL	mixed cache + durable keys
allkeys-random	evict random keys	rarely
volatile-ttl	evict keys with nearest TTL	prioritize short-lived data

PRODUCTION TIP

Decide *what Redis is for* and set the policy to match. **Pure cache?** Set `maxmemory` and `allkeys-lru` (or `lfu`) — running out of memory just evicts cold entries, which is fine.

Holding data you can't lose (queues, locks, sessions you don't want dropped)? Use `noeviction` and alert on memory, because silent eviction of a lock or a queued job is a nasty, hard-to-trace bug. Mixing both in one instance is a common source of "where did my key go?" — separate them if you can.

4.4 Iterating keys safely

You'll need to find or clean up keys by pattern. **Never** use `KEYS` in production — it scans the entire keyspace and blocks the single thread. Use `SCAN`, a cursor that returns keys in small batches.

JAVASCRIPT / NODE.JS

```
// safe pattern iteration (non-blocking)
let cursor = "0";
do {
  const [next, keys] = await redis.scan(cursor, "MATCH", "cache:user:*", "COUNT",
100);
  cursor = next;
  if (keys.length) await redis.del(...keys);
} while (cursor !== "0");
```

PYTHON

```
for key in r.scan_iter(match="cache:user:*", count=100):
    r.delete(key)
```

WHEN NOT TO RELY ON KEY SCANNING

`SCAN` is fine for occasional maintenance, but don't build request-path logic that scans keys to find things — that's using Redis like a database it isn't. If you routinely need "all keys matching X", maintain a **set** of those keys yourself (`SADD index:users user:42`) and read the set. Model your indexes explicitly instead of scanning.

EXERCISE 4.1

Set `maxmemory 50mb` and `allkeys-lru` on a throwaway instance, then write a loop that inserts keys until eviction kicks in; watch `INFO stats evicted_keys` climb and confirm old keys disappear. Separately, prove the `SET` -clears-TTL trap: set a key with `EX 100` , plain- `SET` it, and check `TTL` is now `-1` . Fix it with `KEEPTTL` .

Part 5 · Caching Patterns

Caching is the reason most teams adopt Redis. The idea is simple — keep a hot copy of expensive data — but the failure modes (stale data, stampedes, inconsistency) are where real systems live or die. This part is the patterns and their pitfalls.

5.1 Cache-aside (lazy loading) — the default

Your app checks Redis first; on a miss it loads from the database, stores the result with a TTL, and returns it. This is the pattern you'll use 90% of the time.

JAVASCRIPT / NODE.JS

```
async function getUser(id) {
  const key = `cache:user:${id}`;
  const hit = await redis.get(key);
  if (hit) return JSON.parse(hit); // cache hit

  const user = await db.user.findUnique({ where: { id } }); // miss -> DB
  await redis.set(key, JSON.stringify(user), "EX", 300); // store, 5-min TTL
  return user;
}
```

PYTHON

```
import json
def get_user(id):
    key = f"cache:user:{id}"
    if (hit := r.get(key)) is not None:
        return json.loads(hit)
    user = db_get_user(id)
    r.set(key, json.dumps(user), ex=300)
    return user
```

Strategy	How it works	Trade-off
Cache-aside	app loads on miss, stores with TTL	simple; first request is slow, brief staleness
Write-through	app writes DB <i>and</i> cache together	cache always warm; write latency, complexity
Write-behind	write cache now, DB async later	fast writes; risk of loss, hard to get right
Read-through	a cache layer loads for you	clean API; needs that layer

5.2 Invalidation — the hard part

"There are only two hard things in computer science..." — cache invalidation is one. Your options, simplest first:

- **TTL only:** let entries expire. Simplest; accept bounded staleness.
- **Delete on write:** when the underlying row changes, `DEL` the cache key so the next read repopulates it. Precise, and usually the right default.
- **Update on write:** write the new value into the cache. Fewer misses, but more places to get wrong.
- **Versioned keys:** embed a version in the key (`cache:user:42:v3`); bump the version to invalidate whole groups at once.

JAVASCRIPT / NODE.JS

```
async function updateUser(id, data) {
  const user = await db.user.update({ where: { id }, data });
  await redis.del(`cache:user:${id}`); // invalidate; next read repopulates
  return user;
}
```

COMMON MISTAKE

Updating the database but forgetting to invalidate the cache — now reads serve stale data until the TTL expires, and "it works on my machine but prod shows old data" bugs appear. Make invalidation part of *every* write path (a repository method, a model hook, or a domain event), not something you remember to do by hand at each call site.

5.3 Cache stampede (thundering herd)

When a hot key expires, many concurrent requests miss *at the same instant* and all hammer the database to recompute it — a stampede that can take the DB down exactly when traffic is high. Three defenses, combinable:

JAVASCRIPT / NODE.JS

```
// 1. Lock the recompute: only one request rebuilds; others wait or serve stale
async function getWithLock(key, ttl, loader) {
  const hit = await redis.get(key);
  if (hit) return JSON.parse(hit);

  const lockKey = `lock:${key}`;
  const gotLock = await redis.set(lockKey, "1", "NX", "PX", 5000); // Part 6
  if (!gotLock) { // someone else is rebuilding
    await sleep(50);
    return getWithLock(key, ttl, loader); // retry (or return stale)
  }
  try {
    const value = await loader();
    await redis.set(key, JSON.stringify(value), "EX", ttl);
  }
```

```

    return value;
  } finally {
    await redis.del(lockKey);
  }
}

```

- **Locked recompute** (above): only one caller rebuilds the value.
- **Jittered TTLs**: add randomness (`ttl + random(0..60)`) so keys don't all expire on the same second.
- **Early / probabilistic refresh**: refresh a key slightly *before* it expires, or serve stale-while-revalidate so users never hit an empty cache.

PRODUCTION TIP

Stampede protection only matters for **hot** keys, but for those it's the difference between a smooth cache and a periodic DB meltdown. Add jitter to every cache TTL by default (it's free), and add a recompute lock to the handful of keys you know are hot (homepage, config, popular items).

5.4 What to cache — and what not to

WHEN NOT TO CACHE

Don't cache to hide a query you could make fast with an index — caching adds an invalidation problem you didn't have; an index doesn't. Don't cache data that must always be exactly current (account balances, inventory at checkout) unless you handle invalidation rigorously. And don't cache things that are already fast or rarely read — you're adding moving parts for no win. Cache *expensive, read-heavy, tolerably-stale* data.

EXERCISE 5.1

Implement cache-aside for a "product detail" read with a 5-minute jittered TTL, and delete-on-write invalidation. Then simulate a stampede: fire 100 concurrent reads at the moment the key expires, and add the recompute lock; measure DB calls before (should spike) and after (should be ~1). Do it in whichever of Node/Python you prefer, then port to the other.

Part 6 · Distributed Locks

When several processes might do the same thing at once — generate one report, charge one order, run one cron job across a cluster — you need a **distributed lock**: a flag in Redis that only one holder can own at a time. Redis is a natural fit because `SET NX` is atomic. It's also easy to get subtly wrong, so this part is as much about the pitfalls as the recipe.

6.1 The correct simple lock

The one-liner everyone should memorize: `SET key token NX PX ttl`. `NX` = set only if absent (acquire). `PX ttl` = auto-expire (so a crashed holder can't deadlock the lock forever). The **token** is a unique value that proves *you* hold it.

JAVASCRIPT / NODE.JS

```
import { randomUUID } from "crypto";

async function withLock(name, ttlMs, fn) {
  const token = randomUUID();
  const ok = await redis.set(`lock:${name}`, token, "NX", "PX", ttlMs);
  if (!ok) return false; // someone else holds it
  try {
    return await fn();
  } finally {
    // release ONLY if we still own it (atomic compare-and-delete via Lua)
    await redis.eval(
      `if redis.call('get', KEYS[1]) == ARGV[1]
        then return redis.call('del', KEYS[1]) else return 0 end`,
      1, `lock:${name}`, token,
    );
  }
}
```

PYTHON

```
import uuid
RELEASE = """
if redis.call('get', KEYS[1]) == ARGV[1]
  then return redis.call('del', KEYS[1]) else return 0 end
"""

def with_lock(name, ttl_ms, fn):
  token = str(uuid.uuid4())
  if not r.set(f"lock:{name}", token, nx=True, px=ttl_ms):
    return False
  try:
    return fn()
```

finally:

```
r.eval(RELEASE, 1, f"lock:{name}", token)
```

6.2 Why every piece matters

- **NX** makes acquisition atomic — no check-then-set race.
- **PX (TTL)** prevents a permanent deadlock if the holder crashes before releasing. Without it, one crash freezes the lock forever.
- **The unique token + Lua release** prevents the deadliest bug: releasing *someone else's* lock. If your work runs past the TTL, the lock expires, another process acquires it, and if you then blindly **DEL** on finish you delete *their* lock. The compare-and-delete (only delete if the token is still yours) closes that hole — and it must be atomic, hence Lua.

COMMON MISTAKE

SETNX + a separate **EXPIRE** is **not** safe — the process can die between the two commands and deadlock. Use the single **SET ... NX PX** form. And never release with a plain **DEL key** : always check the token first (Lua), or you'll eventually delete a lock you no longer own once work overruns the TTL.

6.3 TTL, renewal, and honesty about guarantees

Pick a TTL longer than the work should take, but not so long that a crash locks others out for ages. If work can legitimately run long, **renew** the lock periodically (a "watchdog" that extends the TTL while you're alive) rather than setting a huge TTL.

For most apps, a single-instance lock as above is plenty. The **Redlock** algorithm coordinates a lock across multiple independent Redis masters for stronger guarantees, but it's more complex and debated; reach for it only when you genuinely need cross-node consensus.

WHEN NOT TO USE A REDIS LOCK

A Redis lock is a *coordination* tool, not a *correctness* guarantee against all failure modes (clock skew, network partitions, GC pauses can all violate mutual exclusion in edge cases). For operations that must be exactly-once correct — charging a card, decrementing paid inventory — back the lock with an **idempotency key or a database constraint/transaction** so a lock failure can't cause a double-charge. Use the lock to reduce contention; use the DB to guarantee correctness.

PRODUCTION TIP

Reach for a lock to make expensive work run once (report generation, a singleton cron across N app servers, cache recompute — Part 5). Keep the critical section short, always set a TTL, always release with the token check, and prefer the built-in lock helpers (`redis-py` 's `r.lock()` , `ioredis` + a small wrapper or a library like `redlock`) over hand-rolling the Lua every time.

EXERCISE 6.1

Implement `withLock` and use it to guard a "generate monthly report" action. Fire two calls concurrently and confirm only one runs the work. Then simulate an overrun: make the work take longer than the TTL, and prove that the token-checked release does **not** delete the second holder's lock (a plain `DEL` would).

Part 7 · Rate Limiting

Rate limiting protects your app and your third-party bills: N requests per user per minute, M emails per hour, throttled retries against a flaky API. Redis is the standard home because the counting is atomic and shared across every app instance. This part covers the three algorithms you'll actually use.

7.1 Fixed window — simplest

Count requests in the current time bucket; reject over the limit. One `INCR` per request, with a TTL that resets the window.

JAVASCRIPT / NODE.JS

```
async function fixedWindow(userId, limit = 100, windowSec = 60) {
  const bucket = Math.floor(Date.now() / 1000 / windowSec);
  const key = `rl:${userId}:${bucket}`;
  const n = await redis.incr(key);
  if (n === 1) await redis.expire(key, windowSec); // first hit sets the TTL
  return n <= limit;                             // true = allowed
}
```

Cheap and easy, but has a **boundary burst** flaw: a user can send `limit` requests at the end of one window and `limit` more at the start of the next — 2× the limit across the boundary.

7.2 Sliding window — smooth and fair

Use a **sorted set** of request timestamps per user. On each request: drop entries older than the window, count what remains, and add the new one — all atomically. No boundary burst.

JAVASCRIPT / NODE.JS

```
async function slidingWindow(userId, limit = 100, windowMs = 60000) {
  const key = `rl:${userId}`;
  const now = Date.now();
  const pipe = redis.multi();
  pipe.zremrangebyscore(key, 0, now - windowMs); // evict old
  pipe.zadd(key, now, `${now}-${Math.random()}`); // record this request
  pipe.zcard(key);                                // how many in window
  pipe.pexpire(key, windowMs);
  const res = await pipe.exec();
  const count = res[2][1];
  return count <= limit;
}
```

PYTHON

```
def sliding_window(user_id, limit=100, window_ms=60000):
    key = f"rl:{user_id}"
    now = int(time.time() * 1000)
    pipe = r.pipeline()
    pipe.zremrangebyscore(key, 0, now - window_ms)
    pipe.zadd(key, {f"{now}-{random.random()}": now})
    pipe.zcard(key)
    pipe.pexpire(key, window_ms)
    count = pipe.execute()[2]
    return count <= limit
```

7.3 Token bucket — allow bursts, cap the rate

A bucket refills tokens at a steady rate up to a cap; each request spends a token. Allows short bursts while bounding the sustained rate — ideal for throttling calls to an external API. Because it needs read-compute-write done atomically, implement it as a **Lua script** (Part 9) so the whole check-and- consume can't race.

Algorithm	Data structure	Pros	Cons
Fixed window	string <code>INCR</code>	trivial, cheap	boundary bursts (up to 2×)
Sliding window	sorted set	smooth, accurate	more memory per user
Token bucket	hash + Lua	allows bursts, precise rate	needs a script

PRODUCTION TIP

Return the standard signals so clients can behave: on a block, send HTTP **429** with a `Retry-After` header, and expose `X-RateLimit-Remaining`. Key the limit by the right identity — authenticated user id when you have it, IP as a fallback — and set *different* limits for cheap vs expensive endpoints. For anything beyond basics, a battle-tested library (`rate-limiter-flexible` in Node, a middleware in Python) saves you from edge-case bugs.

COMMON MISTAKE

Doing the rate-limit check as separate `GET` then `SET / INCR` from your app — under load, two requests both read "99" and both proceed, blowing the limit. The counting must be atomic: a single `INCR` (fixed window), a pipelined sorted-set update (sliding), or a Lua script (token bucket). Anything that round-trips the count through your app races.

EXERCISE 7.1

Implement fixed-window and sliding-window limiters (10 req / 10s) and hammer each with a burst that straddles a window boundary; observe the fixed window allowing ~2× and the sliding window holding the line. Wire the sliding limiter into an Express or FastAPI middleware that returns 429 + `Retry-After`.

Part 8 · Pub/Sub and Streams

Redis does real-time messaging two very different ways, and picking the right one matters. **Pub/Sub** is fire-and-forget broadcast (no memory). **Streams** are a durable, replayable log with consumer groups (the basis of reliable queues). Node developers: pub/sub is like an `EventEmitter` across the network; streams are like Kafka's little sibling.

8.1 Pub/Sub — broadcast, no durability

Publishers send messages to channels; every currently-connected subscriber gets a copy. Messages are **not stored** — a subscriber that's offline or slow simply misses them.

JAVASCRIPT / NODE.JS

```
// subscriber (dedicated connection – Part 2)
const sub = redis.duplicate();
await sub.subscribe("orders");
sub.on("message", (channel, message) => handleOrder(JSON.parse(message)));

// publisher (normal connection)
await redis.publish("orders", JSON.stringify({ id: 1, total: 999 }));
```

PYTHON

```
sub = r.psubsub()
sub.subscribe("orders")
for msg in sub.listen():
    if msg["type"] == "message":
        handle(json.loads(msg["data"]))
```

NODE.JS MENTAL MODEL

Pub/Sub is a network `EventEmitter`: `publish = emit`, `subscribe = on`. And like an `EventEmitter`, if no one is listening when the event fires, it's gone — there's no buffer, no replay, no acknowledgement. Great for live-but-losable signals (dashboards, presence, cache-invalidation broadcasts); wrong for anything you must not drop.

WHEN NOT TO USE PUB/SUB

Never use Pub/Sub for work that must not be lost — job dispatch, payments, anything where a missed message is a bug. A subscriber restart, a network blip, or a slow consumer drops messages silently. For guaranteed delivery, use **Streams** (below) or a real queue (the Queues cookbook).

8.2 Streams — a durable, replayable log

A stream is an append-only log of entries, each with an auto ID (`<timestamp>-<seq>`). Entries **persist**, can be **re-read**, and — with **consumer groups** — can be distributed across workers with acknowledgements and redelivery of un-acked messages. This is Redis's answer to reliable queues and event sourcing.

SHELL

```
XADD events * type "signup" user "42"      # append (server assigns the ID)
XLEN events
XRANGE events - +                          # read all (replayable)
```

JAVASCRIPT / NODE.JS

```
// producer
await redis.xadd("events", "*", "type", "signup", "user", "42");

// consumer group: create once, then workers read + ack
await redis.xgroup("CREATE", "events", "workers", "$", "MKSTREAM").catch(() => {});
const res = await redis.xreadgroup(
  "GROUP", "workers", "worker-1", "COUNT", 10, "BLOCK", 5000,
  "STREAMS", "events", ">",
);
for (const [, entries] of res ?? []) {
  for (const [id, fields] of entries) {
    await process(fields);
    await redis.xack("events", "workers", id); // acknowledge — removes from
    pending
  }
}
```

Key mechanics: each group tracks a **pending list** of delivered-but-unacked entries; if a worker dies, another can claim them (`XAUTOCLAIM`) so nothing is lost. Cap growth with `XADD ... MAXLEN ~ 100000` (approximate trimming).

Feature	Pub/Sub	Streams
Durability	none (fire-and-forget)	persisted, replayable
Delivery	to connected subscribers only	at-least-once with ack
Multiple workers share load	no (all get all)	yes (consumer groups)
Redelivery on failure	no	yes (pending + claim)
Memory	~none	grows — trim with <code>MAXLEN</code>
Use for	live signals, invalidation	reliable queues, event log

COMMON MISTAKE

Building a job queue on Pub/Sub because it's simpler, then losing jobs whenever a worker restarts. If you need delivery guarantees, use Streams with consumer groups and `XACK`, and remember to `trim` (`MAXLEN`) or the stream grows until it eats memory. (For a full-featured queue, the Queues cookbook's BullMQ/Celery are usually the better tool — they wrap this for you.)

PRODUCTION TIP

Use Pub/Sub for **cache invalidation across instances**: when one app node updates data, `publish` an invalidation so every node drops its local copy. Use Streams when you want a lightweight, durable event bus without standing up Kafka. And always run subscribers/consumers on a **dedicated connection** (Part 2) so blocking reads don't stall your normal commands.

EXERCISE 8.1

(1) Build a pub/sub cache-invalidation broadcast: node A updates a key and publishes `invalidate user:42`; node B (a second client) subscribes and deletes its local copy. (2) Build a stream `events` with a consumer group and two workers; add 20 entries, process and `XACK` them, kill one worker mid-batch, and use `XAUTOCLAIM` to recover its un-acked entries — proving nothing is lost.

Part 9 · Transactions & Lua Scripting

A single Redis command is atomic, but real logic often needs *several* commands to run together without another client sneaking in between. Redis gives you two tools: **transactions** (`MULTI` / `EXEC`) and **Lua scripts** (`EVAL`). Knowing when to use which — and understanding what Redis transactions do and don't guarantee — is the difference between correct and subtly-racy code.

9.1 MULTI/EXEC — queue then run atomically

`MULTI` starts a transaction; commands are *queued*, not executed; `EXEC` runs them all in sequence with **nothing interleaved**. It's how you make several writes land together.

JAVASCRIPT / NODE.JS

```
const res = await redis.multi()
  .set("order:1:status", "paid")
  .incr("stats:paid")
  .zadd("paid:by_time", Date.now(), "order:1")
  .exec(); // all run atomically, in order
```

Two things surprise developers coming from SQL:

- **No rollback.** If one command in the block errors at runtime, the others still execute. Redis transactions guarantee *isolation and ordering*, not all-or-nothing rollback. (Syntax errors abort the whole block; logical errors do not.)
- **No reads-then-decide inside the block.** Because commands are queued, you can't look at a value mid-transaction and branch. For that you need `WATCH` or Lua.

9.2 WATCH — optimistic concurrency

`WATCH` marks keys; if any watched key changes before your `EXEC`, the `EXEC` aborts (returns nil) and you retry. This is optimistic locking — the check-and-set pattern without a held lock.

JAVASCRIPT / NODE.JS

```
async function transfer(from, to, amount) {
  while (true) {
    await redis.watch(`bal:${from}`);
    const bal = Number(await redis.get(`bal:${from}`));
    if (bal < amount) { await redis.unwatch(); return false; }
    const res = await redis
      .multi()
      .decrby(`bal:${from}`, amount)
      .incrby(`bal:${to}`, amount)
```

```

        .exec(); // null if bal:from changed since WATCH -> retry
    if (res) return true;
  }
}

```

9.3 Lua scripts — atomic read-compute-write

For "look at some state, decide, then write" done atomically, a **Lua script** is the cleanest tool. Redis runs the whole script on its single thread with nothing else interleaved — so complex logic (token-bucket rate limiting, conditional updates, the lock release from Part 6) becomes a single atomic operation.

JAVASCRIPT / NODE.JS

```

// atomic "decrement stock only if enough remains" – returns 1 ok, 0 sold out
const BUY = `
  local stock = tonumber(redis.call('get', KEYS[1]) or '0')
  if stock >= tonumber(ARGV[1]) then
    redis.call('decrby', KEYS[1], ARGV[1])
    return 1
  end
  return 0`;
const ok = await redis.eval(BUY, 1, "stock:sku42", 2);

```

PYTHON

```

BUY = """
local stock = tonumber(redis.call('get', KEYS[1]) or '0')
if stock >= tonumber(ARGV[1]) then
  redis.call('decrby', KEYS[1], ARGV[1]); return 1
end
return 0"""
ok = r.eval(BUY, 1, "stock:sku42", 2)

```

Use `SCRIPT LOAD` + `EVALSHA` to cache scripts by hash and avoid resending the body each call (ioredis: `defineCommand`; redis-py: `r.register_script`).

Need	Tool
Several writes, atomic + ordered	<code>MULTI / EXEC</code>
Retry if a key changed underneath	<code>WATCH</code> + <code>MULTI</code>
Read → decide → write, atomically	Lua (<code>EVAL</code>)
A single conditional counter/limiter	Lua

COMMON MISTAKE

Expecting `MULTI / EXEC` to roll back on error like a SQL transaction — it doesn't. If you need "do all of this only if a condition holds," that's a Lua script (check the condition inside the script) or `WATCH`, not a bare `MULTI`. Also: keep Lua scripts **fast and non-looping over huge data** — they block the single thread for their whole duration, so a slow script freezes the whole server.

WHEN NOT TO SCRIPT

Don't push business logic into Lua because you can. Scripts are harder to test, debug, and observe than app code, and they run inside the database. Reserve Lua for the narrow case it's *uniquely* good at: a small, hot, must-be-atomic read-compute-write (rate limiter, conditional stock decrement, safe lock release). Everything else belongs in your app.

EXERCISE 9.1

Implement the "buy if in stock" Lua script and hammer it with concurrent buyers against a stock of 5 — assert exactly 5 succeed and the rest get 0, with no oversell. Then implement `transfer` with `WATCH / MULTI` and force a concurrent modification to trigger a retry. Do both in Node and Python.

Part 10 · Persistence & Durability

Redis is in-memory, but it can write to disk so a restart doesn't lose everything. Understanding the two persistence modes — and their honest guarantees — tells you how much you can trust Redis with, and how to configure it for your use case.

10.1 RDB vs AOF

Mode	What it is	Recovery	Trade-off
RDB	point-in-time snapshots of the dataset	fast to load	can lose everything since the last snapshot
AOF	append-only log of every write command	replay on start	larger files, slightly slower writes, minimal loss
Both	AOF for durability + RDB for fast restarts	best of both	more disk/IO
None	pure memory (<code>save ""</code> , <code>appendonly no</code>)	nothing survives	fastest; a cache you can lose

SHELL

```
# redis.conf highlights
save 900 1           # RDB: snapshot if ≥1 key changed in 900s (and other rules)
appendonly yes       # AOF on
appendfsync everysec # fsync the AOF once per second (the usual balance)
```

`appendfsync` is the durability dial: `always` (fsync every write — safest, slowest), `everysec` (up to ~1s of writes at risk — the sane default), `no` (let the OS decide — fastest, least safe).

10.2 The honest durability story

Even with AOF `always`, Redis is **not** a database you should treat as a source of truth. Replication is asynchronous, failover can lose the last writes, and `everysec` risks a second of data on a crash. Design so that a lost Redis is survivable: it should hold data that is *derived* (rebuildable from the DB), *transient* (sessions, rate-limit counters), or *reconstructable* — not the only copy of something important.

WHEN NOT TO RELY ON PERSISTENCE

Don't use Redis persistence as a reason to make it your primary store for critical data. If the correct answer to "what if we lose the last few seconds of writes?" is "that's an incident," that data belongs in Postgres, with Redis as a cache or accelerator in front. Persistence is a faster-restart and lower-blast-radius feature, not a promise of zero loss.

10.3 Choosing per use case

- **Pure cache** (Part 5): persistence off or RDB-only is fine — on restart the cache repopulates from the DB. Prioritize memory and speed.
- **Sessions / rate limits**: AOF `everysec` — losing a second is usually acceptable, and you avoid logging everyone out on a restart.
- **Queues / streams you can't lose**: AOF `everysec` **and** replication (Part 11), plus idempotent consumers so a redelivered or lost-then-retried message is safe — durability at this layer is defense-in-depth, not a guarantee.

10.4 Memory is the real constraint

Because everything lives in RAM, watch memory the way you'd watch disk on a database. `INFO memory` shows `used_memory` and fragmentation; set `maxmemory` and an eviction policy (Part 4). Large values, unbounded lists/streams, and keys without TTLs are the usual causes of creeping memory.

PRODUCTION TIP

Set `maxmemory` to leave headroom (Redis needs extra RAM during RDB saves and AOF rewrites — a fork briefly copies memory), enable AOF `everysec` for anything you'd rather not lose, and monitor `used_memory`, `mem_fragmentation_ratio`, and `evicted_keys`. On managed Redis (ElastiCache, Upstash, Redis Cloud) these are dashboards; on self-hosted, scrape `INFO`.

COMMON MISTAKE

Running a single Redis with no persistence and no replica as the store for something you actually care about — one restart or crash and it's gone. At minimum, match persistence to the data's importance, and for critical workloads add a replica (Part 11). Conversely, forcing AOF `always` on a pure cache just wastes IO for no benefit.

EXERCISE 10.1

On a throwaway instance: set several keys, `SAVE`, restart the container, and confirm they survive with RDB. Then disable RDB, enable AOF `everysec`, write keys, `kill -9` the process, restart, and observe what survived. Write one paragraph on which persistence setting you'd choose for (a) a cache, (b) session storage, and (c) a job queue — and why.

Part 11 · Production Redis

Running Redis for real adds concerns beyond commands: high availability, scaling, security, and observability. This part is the operational checklist — what to set up, what to watch, and the mistakes that cause 2 a.m. pages.

11.1 High availability: replication, Sentinel, Cluster

A single Redis is a single point of failure. Three levels of resilience:

Setup	What it gives you	Use when
Replication (replica of primary)	read scaling + a warm standby	almost always, as a baseline
Sentinel	automatic failover to a replica	you need HA without sharding
Cluster	sharding across nodes + HA	dataset/throughput exceeds one node

Replicas are asynchronous — a failover can lose the last writes (Part 10's honesty applies). **Cluster** shards keys across nodes by hash slot; the catch is that multi-key operations (and `MGET`, transactions, Lua touching several keys) only work when the keys live on the same shard. Use **hash tags** — `{user:42}` — to force related keys onto one slot.

PRODUCTION TIP

Most teams should **use managed Redis** (ElastiCache, Memystore, Upstash, Redis Cloud) rather than operating Sentinel/Cluster themselves — failover, backups, and upgrades are genuinely hard to run well. Start with a primary + one replica; move to Cluster only when a single node's memory or throughput is the proven bottleneck, and design keys with hash tags *before* you shard.

11.2 Security

Redis is fast partly because it assumes a trusted network — so securing it is *your* job:

- **Never expose Redis to the public internet.** Bind to private interfaces; put it behind a VPC/firewall. An open Redis is a well-known, actively-scanned way to get owned.
- **Require auth** (`requirepass`, or ACL users in Redis 6+) and use TLS in transit on untrusted networks.
- **Rename or disable dangerous commands** (`FLUSHALL`, `CONFIG`, `KEYS`, `DEBUG`) in shared environments via ACLs.
- Keep secrets (the Redis URL/password) in env/secret storage, never in code.

11.3 Observability

Watch these signals (via `INFO`, `redis-cli`, or your provider's dashboard):

Signal	Command / field	Why
Memory used / fragmentation	<code>INFO memory</code>	OOM and eviction risk
Evictions	<code>evicted_keys</code>	cache too small or a leak
Hit rate	<code>keyspace_hits/misses</code>	is the cache actually helping?
Slow commands	<code>SLOWLOG GET</code>	the single thread being blocked
Connected clients	<code>INFO clients</code>	connection leaks
Ops/sec, latency	<code>INFO stats</code> , <code>--latency</code>	saturation
Replication lag	<code>INFO replication</code>	failover data-loss window

11.4 The production mistakes that bite

COMMON MISTAKE

The recurring Redis outages come from a short list: (1) a `KEYS *` or big `SMEMBERS / SORT` blocking the single thread under load — use `SCAN` and bounded ranges; (2) no `maxmemory` / eviction policy, so Redis OOMs or the OS kills it; (3) keys without TTLs slowly filling memory; (4) exposing Redis to the internet with no auth; (5) big values / hot keys creating latency spikes. Almost every incident is one of these.

WHEN NOT TO ADD REDIS COMPLEXITY

Don't jump to Cluster, Sentinel, and multi-region before you need them. A single well-monitored primary + replica handles a lot of load. Added topology is added failure modes and operational burden — scale it when metrics demand it, not preemptively.

EXERCISE 11.1

Set up a primary + replica locally (two containers, `replicaof`), write to the primary, and confirm reads on the replica. Kill the primary and observe the replica's state. Then run through the security checklist against your dev setup: is it bound to localhost, is auth on, is `FLUSHALL` reachable? Finally, trigger a `SLOWLOG` entry with a deliberately slow command and read it back.

Part 12 · Patterns & Commands Cheat Sheet

Keep this at your desk. Data types → use, commands by task, and the pattern recipes from this book in one place.

12.1 Data type → what it's for

Type	Use for	Key commands
String	values, counters, flags, cache blobs	SET GET INCR SETNX MGET
Hash	objects/records, partial updates	HSET HGET HINCRBY HGETALL
List	queues, stacks, recent feeds	LPUSH RPOP BRPOP LRange LTRIM
Set	uniqueness, membership, tags	SADD SISMEMBER SCARD SINTER
Sorted set	leaderboards, priority, time index	ZADD ZRANGEBYSCORE ZREVRANGE ZRANK
Bitmap	per-user booleans at scale	SETBIT GETBIT BITCOUNT
HyperLogLog	approx unique counts, tiny memory	PFADD PFCOUNT
Stream	durable log, reliable queues	XADD XREADGROUP XACK XAUTOCLAIM

12.2 Commands by task

Task	Command(s)
Set with TTL	SET k v EX 60
Keep TTL on update	SET k v KEeptTL
Atomic counter	INCR / INCRBY / HINCRBY
Acquire a lock	SET lock:x token NX PX 5000
Release a lock safely	Lua compare-and-DEL on the token
Multi-get / multi-set	MGET / MSET
Batch round trips	pipeline / MULTI
Iterate keys safely	SCAN (never KEYS in prod)
Expire / inspect TTL	EXPIRE / TTL / PERSIST
Atomic read-decide-write	EVAL (Lua)
Optimistic transaction	WATCH + MULTI / EXEC
Delayed job / time index	ZADD due <ts> + ZRANGEBYSCORE
Broadcast (losable)	PUBLISH / SUBSCRIBE

Task	Command(s)
Reliable stream	<code>XADD</code> + consumer group + <code>XACK</code>
Inspect	<code>INFO</code> <code>SLOWLOG</code> <code>GET</code> <code>MEMORY</code> <code>USAGE</code> <code>DBSIZE</code>

12.3 Node ↔ Python ↔ CLI

Task	ioredis (Node)	redis-py (Python)
Connect	<code>new Redis(url)</code>	<code>redis.from_url(url, decode_responses=True)</code>
Set + TTL	<code>set(k,v,"EX",60)</code>	<code>set(k,v,ex=60)</code>
Pipeline	<code>redis.pipeline()...exec()</code>	<code>r.pipeline()...execute()</code>
Transaction	<code>redis.multi()...exec()</code>	<code>r.pipeline(transaction=True)</code>
Lua	<code>redis.eval(src,n,...)</code>	<code>r.eval(src,n,...)</code> / <code>register_script</code>
Pub/sub	<code>redis.duplicate().subscribe()</code>	<code>r.psubsub()</code>
Scan	<code>redis.scan(cursor,...)</code>	<code>r.scan_iter(match=...)</code>
Lock helper	<code>redlock</code> / wrapper	<code>r.lock(name, timeout=...)</code>
Async	native	<code>redis.asyncio</code>

12.4 The pattern recipes (from this book)

- **Cache-aside** (Part 5): `GET` → miss → load DB → `SET ... EX ttl` with jitter; invalidate with `DEL` on write.
- **Stampede guard** (Part 5): recompute behind a `SET NX` lock + jittered TTL.
- **Safe lock** (Part 6): `SET k token NX PX ttl`; release via Lua token check.
- **Rate limit** (Part 7): fixed = `INCR` + `EXPIRE`; sliding = sorted set; bursty = token bucket in Lua. Return 429 + `Retry-After`.
- **Delayed work** (Part 3/8): sorted set scored by run-at; poll `ZRANGEBYSCORE 0 now`.
- **Reliable events** (Part 8): stream + consumer group + `XACK` + `XAUTOCLAIM`; trim with `MAXLEN`.
- **Atomic conditional write** (Part 9): a small Lua script.

12.5 The production checklist

- [] `maxmemory` + an eviction policy chosen for the workload (Part 4)
- [] TTLs on all cache keys; jitter on hot ones (Parts 4-5)
- [] AOF `everysec` for data you'd rather not lose (Part 10)
- [] Primary + replica (managed Redis preferred) (Part 11)
- [] Not exposed to the internet; auth on; TLS on untrusted nets (Part 11)
- [] No `KEYS` /big blocking commands on the hot path; `SCAN` instead
- [] Dedicated connections for pub/sub and blocking reads (Part 2)

- [] Monitoring: memory, evictions, hit rate, slowlog, replication lag

YOU'RE READY

Redis rewards a small amount of discipline with enormous leverage: the same handful of structures and the atomicity guarantee cover caching, locks, rate limiting, real-time messaging, and queues. Master the patterns here and you'll reach for Redis correctly — and know the cases where a database or a dedicated queue is the better tool. The **Queues** cookbook builds directly on Parts 6–9; the **Mailpit** cookbook and the **Backend Infra Sprint** plan complete the set.